

Mecha Connect

Architecture & Enterprise Engineering Standards Overview

Engineering Leadership Team

Version 1.0

Abstract

This document serves as the architectural summary and technical standards overview for **Mecha Connect**, an AI-Powered Roadside Assistance and Vehicle Services Platform. It outlines the system architecture, core engineering principles, technology stack, and security protocols designed to scale to millions of users while maintaining sub-second latency and offline-first reliability.

Contents

1	Engineering Overview & Vision	3
1.1	Mission	3
1.2	Core Engineering Principles	3
1.3	Assumptions, Constraints, and Non-Goals	3
1.4	Architecture Decision Records (ADRs) & Technical Debt	3
1.5	Coding Standards	3
2	System Architecture	4
2.1	System Context Diagram	4
2.2	Technology Stack & Rationale	4
2.3	Scalability Strategy & High-Level Data Flow	4
3	API Standards & Communication	5
4	Database Standards & Data Modeling	5
4.1	Geospatial ER Modeling	5
4.2	Indexing, Partitioning, and Soft Deletes	5
4.3	Backups & Disaster Recovery (DR)	6
5	Artificial Intelligence Integration	6
5.1	RAG Pipeline & Vector Architecture	6
5.2	Latency Optimization & Caching	6
5.3	AI Cost Optimization & Token Budgets	6
5.4	Guardrails & Evaluation	6
6	Security, Compliance & Identity	6
6.1	Authentication & RBAC	6
6.2	OWASP Mitigations & Secrets Management	7
6.3	Compliance: PCI-DSS, GDPR, & SOC 2 Readiness	7
6.4	Pre-Deployment Security Checklist	7

6.5	Audit Logging	7
7	DevOps, Observability & Reliability	7
7.1	Git Workflow & CI/CD Pipeline	7
7.2	Blue-Green Deployments & Canary Releases	7
7.3	Observability & Incident Management	8
7.4	Performance SLAs & KPIs	8
8	Future Engineering Roadmap	8

1 Engineering Overview & Vision

1.1 Mission

Mecha Connect's engineering organization exists to build the most reliable, fast, and trustworthy roadside assistance platform on the planet. Our systems are engineered to function flawlessly during emergencies, in network dead zones, under extreme load, and across diverse device capabilities. We do not just build features; we build lifelines.

1.2 Core Engineering Principles

- **Offline-First by Default:** Roadside emergencies happen in network dead zones. All critical flows possess offline fallbacks, with deterministic conflict resolution upon reconnection.
- **Fail Gracefully:** Every API call has error handling, and every failure mode has a designed fallback. Users are presented with intuitive recovery steps, not technical stack traces.
- **Build for Urgency:** Every interaction is optimized for speed. SOS features are accessible in a single tap, and service requests complete in under three taps.
- **Security is Non-Negotiable:** All data in transit utilizes TLS 1.3, sensitive data at rest is AES-256 encrypted, and payment data never directly touches our application servers.

1.3 Assumptions, Constraints, and Non-Goals

- **Assumptions:** High mobile network volatility for mechanics in rural areas; GPS jitter is expected and must be smoothed via Kalman filtering on the client.
- **Constraints:** 500ms hard latency ceiling for SOS and primary booking APIs. The application must consume $< 50\text{MB}$ of memory to run on low-end Android devices.
- **Non-Goals:** We do not own a fleet of vehicles; we are purely a matching, routing, and payment platform. We will not build a custom maps engine (relying on Mapbox/Google Maps).

1.4 Architecture Decision Records (ADRs) & Technical Debt

All major architectural shifts, framework adoptions, and security protocols must be documented as immutable ADRs in the monorepo. Technical debt is tracked via the `tech-debt` tag in Jira, with a strict **15% engineering capacity budget** allocated per sprint for continuous refactoring and dependency upgrades.

1.5 Coding Standards

Code readability and maintainability are prioritized over cleverness.

- **Single Responsibility:** One file, one class, one function—one job.
- **Strict Typing:** All Python backends mandate strict `mypy` type hints. Dart/Flutter requires sound null safety.
- **Immutability:** State changes favor immutable models (e.g., `freezed` in Flutter, `Pydantic` in Python).
- **Linting Gates:** Code must pass `ruff` (Python) and `dart analyze` with zero warnings before merging.

2 System Architecture

The Mecha Connect ecosystem is built on a modern, decoupled microservices architecture utilizing a Monorepo strategy (via Melos) for unified deployment and versioning.

2.1 System Context Diagram

```
1 graph TD
2   Client[Flutter Mobile/Web Apps] --> AGW[FastAPI API Gateway]
3   AGW --> Auth[Auth Service]
4   AGW --> Core[Booking & Payment Services]
5   AGW --> Notif[Notification Service]
6
7   Core --> DB[(PostgreSQL + PostGIS)]
8   Core --> Cache[(Redis Cluster)]
9   Core --> Stripe[Stripe API]
10
11  AGW --> AI[AI Matching & Assistant]
12  AI --> Pinecone[(Pinecone Vector DB)]
13  AI --> OpenAI[OpenAI LLM]
```

Listing 1: Mecha Connect System Context (Mermaid Architecture)

2.2 Technology Stack & Rationale

Layer	Technology	Rationale
Frontend	Flutter, BLoC	Write-once native compilation for iOS/Android; robust state management.
API Gateway	FastAPI (Python)	Native async I/O, Pydantic validation, and auto-generated OpenAPI docs.
Microservices	FastAPI, Celery	High-throughput async endpoints with Celery for heavy background tasks.
Primary DB	PostgreSQL 16	ACID compliance for payments; PostGIS handles complex geospatial routing.
Cache/Queue	Redis 7	Sub-millisecond latency for session management, rate limiting, and pub/sub.
AI Engine	Pinecone, OpenAI	Vector search for RAG; LLMs for semantic matching and intelligent chat.
Infrastructure	AWS ECS, Terraform	Immutable IaC deployments with Fargate for zero-maintenance container scaling.

Table 1: Mecha Connect Core Technology Stack

2.3 Scalability Strategy & High-Level Data Flow

Our architecture is designed for horizontal scalability and high availability.

1. **Stateless Computing:** All microservices are entirely stateless. Session state is offloaded to Redis, allowing AWS ECS to scale instances horizontally based on CPU load and request queues.

2. **Data Flow:** The API Gateway validates JWTs and enforces rate limits before routing to specific domain services.
3. **Read/Write Segregation:** Heavy read operations (reporting, historical bookings) are routed to PostgreSQL read replicas, protecting the primary writer node.

3 API Standards & Communication

- **RESTful Conventions:** All APIs follow strict REST principles, versioned via URI (e.g., `/api/v1/bookings`).
- **Pagination:** Cursor-based pagination is utilized for high-performance feed scrolling, while offset-based pagination is reserved for admin portals.
- **Contract-Driven Development:** OpenAPI 3.0 specs are auto-generated from Pydantic models. Client SDKs are generated directly from this spec to ensure type safety across the network boundary.

4 Database Standards & Data Modeling

4.1 Geospatial ER Modeling

PostGIS handles real-time location tracking. Mechanic and Customer locations are stored as `GEOGRAPHY(Point, 4326)` columns, mapped relationally to Users and Vehicles via UUID v7 primary keys.

```
1 CREATE TABLE users (  
2     id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     email       VARCHAR(255) NOT NULL UNIQUE,  
4     phone_number VARCHAR(20) NOT NULL UNIQUE,  
5     password_hash VARCHAR(255) NOT NULL,  
6     role        VARCHAR(20) NOT NULL DEFAULT 'customer'  
7     CHECK (role IN ('customer', 'mechanic', 'admin')),  
8     is_active   BOOLEAN NOT NULL DEFAULT TRUE,  
9     deleted_at  TIMESTAMPTZ,  
10    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
11 );  
12 CREATE INDEX idx_users_is_active ON users(is_active) WHERE is_active = TRUE;
```

Listing 2: Core User Schema Implementation

4.2 Indexing, Partitioning, and Soft Deletes

- **Indexing:** Every foreign key and frequent WHERE clause is indexed. We utilize GIST indexes for geospatial queries and B-tree partial/composite indexes for standard lookups.
- **Soft Deletes:** Hard deletions are prohibited for core entities. We utilize a `deleted_at` timestamp managed via SQLAlchemy mixins.
- **Partitioning:** Time-series data (e.g., wallet transactions, high-volume logs) is declaratively partitioned by month in PostgreSQL to maintain fast insert/read times.
- **Schema Migrations:** Alembic manages version-controlled migrations. All migrations must include both `upgrade()` and `downgrade()` methods.

4.3 Backups & Disaster Recovery (DR)

Continuous WAL archiving streams directly to AWS S3, providing Point-in-Time Recovery (PITR). Automated, encrypted snapshots are taken daily. Our Recovery Point Objective (RPO) is < 5 minutes, and Recovery Time Objective (RTO) is < 4 hours.

5 Artificial Intelligence Integration

Mecha Connect utilizes a sophisticated Retrieval-Augmented Generation (RAG) pipeline to power mechanic matching and intelligent customer assistance.

5.1 RAG Pipeline & Vector Architecture

- **Embeddings:** User text inputs ("My car is smoking") are converted to vector embeddings using `text-embedding-3-small`.
- **Retrieval:** Embeddings are matched via Cosine Similarity in Pinecone against mechanic history, skill sets, and vehicle repair manuals.
- **Generation:** OpenAI LLMs synthesize the retrieved context to either assign a matching score to mechanics or provide troubleshooting advice to the user.

5.2 Latency Optimization & Caching

To maintain a strict latency budget (< 2s P95 for matching), we implement **Semantic Caching**. Queries are embedded and checked against a Redis semantic cache; queries with a similarity threshold of > 0.95 return cached responses, bypassing the LLM entirely and reducing costs. AI chat features utilize Server-Sent Events (SSE) streaming for a Time-To-First-Byte (TTFB) of < 500ms.

5.3 AI Cost Optimization & Token Budgets

To ensure sustainable unit economics at scale, we enforce strict token budgets (max 4,000 tokens per interaction) and dynamic context window truncation. The Semantic Cache targets a 40% cache hit rate for common troubleshooting queries. Cost-per-1k-inferences is tracked as a primary engineering KPI on our Grafana dashboards.

5.4 Guardrails & Evaluation

AI outputs pass through strict programmatic guardrails. Content guardrails block PII transmission, and Safety guardrails prevent unsafe mechanical advice (e.g., hallucinated pricing), routing high-risk queries to human dispatchers. Our automated evaluation pipeline checks model outputs against a gold-standard dataset, requiring > 90% accuracy and < 5% hallucination rates before deployment.

6 Security, Compliance & Identity

6.1 Authentication & RBAC

- **JWT Architecture:** Authentication uses asymmetric JWT (RS256). Short-lived access tokens are paired with rotating refresh tokens stored in HTTP-only cookies.

- **Role-Based Access Control:** A strict hierarchy (Customer, Mechanic, Admin) governs API access, enforced by FastAPI dependencies on every endpoint.

6.2 OWASP Mitigations & Secrets Management

The platform natively mitigates OWASP Top 10 vulnerabilities. SQL injection is prevented via SQLAlchemy parameterized queries; XSS is prevented by strictly returning JSON data; and robust rate limiting (Redis) prevents abuse. **Secrets Management:** No secrets are stored in code. Environment variables pull from AWS Secrets Manager at runtime via Pydantic BaseSettings.

6.3 Compliance: PCI-DSS, GDPR, & SOC 2 Readiness

Mecha Connect operates under a shared responsibility model. Payment processing is fully off-loaded to Stripe via Stripe Elements, keeping our infrastructure strictly out of PCI scope (SAQ A compliance). PII (Personally Identifiable Information) and geolocation data are strictly isolated, supporting automated GDPR "Right to be Forgotten" cascade deletions. The architecture and audit logging layers are designed to support upcoming SOC 2 Type II audits.

6.4 Pre-Deployment Security Checklist

Before any major release, the CI/CD pipeline enforces a strict security gate:

- Automated SAST/DAST scans (SonarQube/OWASP ZAP).
- Dependency vulnerability and license checks (Trivy).
- Exposed secrets scan (TruffleHog) blocking PR merges.
- Verified AWS IAM least-privilege role assertions.

6.5 Audit Logging

All sensitive data mutations (financial, PII) are tracked in an immutable `audit_log` table that records `old_values`, `new_values`, user ID, and timestamps, ensuring compliance and forensic capability.

7 DevOps, Observability & Reliability

7.1 Git Workflow & CI/CD Pipeline

We operate on **Trunk-Based Development**. Feature branches are short-lived and merged via squash commits to `main`. Every Pull Request triggers GitHub Actions that run:

1. **Static Analysis:** `ruff` (Python) and `dart analyze`.
2. **Testing:** Unit, integration, and widget tests (target 85%+ coverage).
3. **Security Scanning:** Trivy and Dependabot dependency vulnerability scans.

7.2 Blue-Green Deployments & Canary Releases

Production deployments utilize a **Blue-Green Strategy** on AWS ECS. Traffic is shifted gradually via Canary releases (5% → 25% → 100)

7.3 Observability & Incident Management

- **Logging:** Structured JSON logging with injected `X-Correlation-ID` headers traces requests across all microservices via Loki/ELK.
- **Incident Matrix:** Incidents trigger automated PagerDuty alerts based on severity (SEV1–SEV4). SEV1 (Critical Outage) carries a 15-minute response SLA.

7.4 Performance SLAs & KPIs

Operation	P95 Target	Degraded Threshold
Standard API (DB Read)	< 300ms	> 1s
Standard API (DB Write)	< 500ms	> 2s
AI Inference	< 1s	> 3s
App Startup (Cold)	< 3s	> 7s

Table 2: System Performance Service Level Objectives (SLOs)

8 Future Engineering Roadmap

To continue scaling globally, our upcoming technical milestones include:

- **Multi-Region Active-Active Deployments:** Expanding AWS footprint to support cross-region failover and lower latency for global users.
- **Edge Computing AI:** Pushing lightweight LLMs directly to the Flutter application for offline emergency triage capabilities.
- **Event-Driven Billing:** Migrating from synchronous API payments to an Apache Kafka-driven event stream for complex, dynamic pricing models.